C++语言程序设计

第0章 绪论

课程成绩

1.C++背景知识

兼容C、数据抽象、面向对象、泛型编程

指令式编程、面向对象编程、函数式编程、需要编译的语言

程序的开发过程：编辑、编译、连接、运行调试

第1章 C++新特征

1.C++对C的补充

iostream 输入输出流

- 对象endl:end of line，换行
- 对象cout:输出流对象
- 对象cin:输入流对象，一般就是键盘
- `<<` 插入运算符
- `>>` 提取运算符

命名空间

`std::` 表示定义在命名空间std中 `::` 作用域操作符

可以直接在开头添加： `using namespace std;` 之后就不用 `std::`

💡 注意注释：/* */不可嵌套使用

函数重载

函数名相同，但参数列表不同，返回类型可相同可不同

```cpp
// 函数重载示例
#include <iostream>
using namespace std;

// 计算整数的平方
int square(int x) {
    return x * x;
}

// 计算浮点数的平方
double square(double x) {
    return x * x;
}

// 计算两个整数的和
int add(int a, int b) {
    return a + b;
}

// 计算三个整数的和
int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    cout << "整数5的平方: " << square(5) << endl;
    cout << "浮点数2.5的平方: " << square(2.5) << endl;
    cout << "2 + 3 = " << add(2, 3) << endl;
    cout << "2 + 3 + 4 = " << add(2, 3, 4) << endl;
    return 0;
}

```

上面的代码展示了函数重载的两个例子：

- `square()` 函数：一个版本接受整数参数，另一个版本接受浮点数参数
- `add()` 函数：一个版本接受两个参数，另一个版本接受三个参数

编译器会根据传入的参数类型和数量自动选择合适的函数版本。这就是函数重载的核心概念。

函数模板

```cpp
#include <iostream>
using namespace std;

// 定义一个函数模板，T 是类型参数
template <typename T>
T add(T a, T b) {
    return a + b;
}

int main() {
    cout << add(3, 4) << endl;        // int 版本：3 + 4
    cout << add(3.5, 2.1) << endl;    // double 版本：3.5 + 2.1
    cout << add(string("Hi "), string("there")) << endl; // string 版本
}
```

默认参数

如果没有传入这个参数，就用设好的值，语法：返回类型 函数名(参数 = 默认值, ...);

引用

对象的别名，实际应用中主要用作函数的形式参数

```cpp
int a;
int &b = a;
int &c = b;
// 变量a有两个别名b和c
```

引用就像个“外号”，引用一定义就必须绑定上一个变量，并且一旦绑定就不能再修改指向另一个变量

常见用途：函数传参，避免拷贝（代替C中传参使用指针的写法）

- C的写法（用指针）

```cpp
#include <stdio.h>

void swap(int *a, int *b) {
    int tmp;
    tmp = *a;
```

```
    *a = *b;
    *b = tmp;
}

int main() {
    int a = 4, b = 6;
    swap(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}

```

- C++的写法（用引用）

```
#include <iostream>
using namespace std;

void swap(int &a, int &b) {
    int tmp;
    tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int a = 4, b = 6;
    swap(a, b);
    cout << a << " " << b << endl;
    return 0;
}

```

💡

常量引用初始化规则

- **普通引用**：必须绑定到已有变量，不能绑定临时值或常量。

```
int& r = 5;    // ❌
```

- **const 引用**：可以绑定临时值、常量、类型转换后的临时对象。

```
const int& r1 = 5;      // ✓
const int& r2 = i + 3;  // ✓
const int& r3 = d;      // ✓ double → int 临时量
```

- 编译器会生成一个临时变量，引用指向它。
- 临时对象不可修改，所以必须是 `const` 。

内联函数

在定义函数的最前面加inline，或者在类里的成员函数默认就是inline

- 普通函数：调用时要跳转 → 有调用开销。
- **内联函数**：编译器在调用点直接把函数体"展开"，省掉函数调用的跳转和返回。

作用域

- 花括号内{}的变量，只能在括号内使用
- 作用域运算符::可以访问全局变量（在全局变量和局部变量同名时使用）

字符串变量

- 数据类型：string，可直接赋值一个字符串，可直接用cin或cout
- 字符串连接操作： `+`

```
string w1 = "C++ ";
string w2 = "Language";
string result = w1 + w2;
cout << result << endl;   // C++ Language
```

- 字符串比较（ `==` , `<` , `>` 等）

```
string a = "apple";
string b = "banana";

cout << (a == b) << endl;  // 0 (false)
cout << (a < b) << endl;   // 1 (true, 因为 apple < banana 按字典序)
```

- 字符操作（取字符 / 改字符） 直接当数组进行操作

```cpp
string s = "Cat";
cout << s[0] << endl;   // C
s[0] = 'B';
cout << s << endl;      // Bat
```

new 与 delete

- **new**：分配内存并初始化，返回指针，会调用构造函数。
- **delete**：释放 `new` 的内存，会调用析构函数。
  - 单对象：`delete p;`
  - 数组：`delete[] p;`

```cpp
#include <iostream>
using namespace std;

int main() {
    int* p = new int(42);     // 分配一个 int，值为 42
    cout << *p << endl;       // 输出 42
    delete p;                 // 释放

    int* arr = new int[3];    // 分配数组
    for (int i = 0; i < 3; i++) arr[i] = i+1;
    for (int i = 0; i < 3; i++) cout << arr[i] << " "; // 1 2 3
    delete[] arr;             // 释放数组
}
```

第2章 C++面向对象编程

0 面向对象程序设计方法概述

- 几个概念
  - 抽象 abstraction:类class
  - 封装encapsulation与信息隐蔽information hiding
  - 继承inheritance与重用software reusability
  - 多态性polymorphism

- 类和对象
  - 类是对象的抽象，不占内存
  - 对象是类的示例，占内存

1 类的申明和对象的定义

声明类的类型（C++）

- **类头（class head）**：

```
class 类名 { ... };
struct 类名 { ... };
```

- **类体（class body）**：类的成员表（数据成员 + 成员函数）。
- **访问控制默认值**：
  - `class` 默认是 **private**（私有）。
  - `struct` 默认是 **public**（公有）。

示例

struct（默认 public）：

```
struct Student {
    int num;
    char name[20];
    char sex;
};

Student stud1, stud2;
```

class（默认 private）：

```
class Student {
    int num;
    char name[20];
    char sex;
    void display() {
        cout << "num:" << num << endl;
        cout << "name:" << name << endl;
        cout << "sex:" << sex << endl;
    }
};
```

带 **访问限定符** 的 class：

```
class Student {
private:
    int num;
    char name[20];
    char sex;
public:
    void display() {
        cout << "num:" << num << endl;
        cout << "name:" << name << endl;
        cout << "sex:" << sex << endl;
    }
};
```

限定符	本类	派生类	类外
private	✅	❌	❌
protected	✅	✅	❌
public	✅	✅	✅

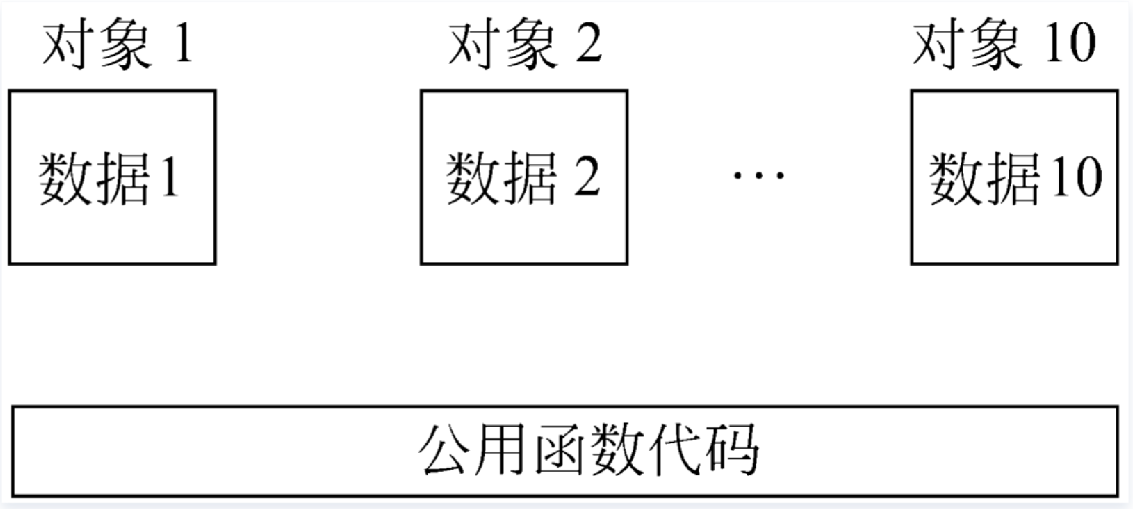
2. 类的成员函数

- 也可指定为private\protected\public
- 在类外定义成员函数：必须先在类体中作原型声明
  - 在类体中作原型声明：`类型 函数名（形参表）；`
  - 在类外定义成员函数：`类型 类名::函数名(形参表) { ... };`
  - `::` 作用域限定符

- 内置成员函数（inline成员函数）
  - 如果函数定义 **直接写在类体里**，且不包含循环/复杂控制结构，编译器会**自动认为它是 inline**。
  - 如果成员函数写在类外定义，要想变成内联函数，就必须显式加 `inline` 。

3. 成员函数的存储方式

- 实际上，**对象里只存数据**，并不存函数代码。
- 函数代码在内存中只放一份，所有对象都共用。
- 不同对象在调用同一个成员函数时，通过 `this` 指针来区分“我是谁”。



4. 访问对象的成员

- 通过对象名和成员运算符
  - 格式：对象名.成员名
  - 只有成员函数能访问类中所有成员，类外只能访问public成员
- 通过指向对象的指针

```
class Time{...};
Time t, *p;
p = &t; //p是指针
```

三种同义的方式： `p→hour` `(*p).hour` `t.tour`

- 通过对象的引用
  - 对象也可以定义别名，使用方法完全相同

6. 类的封装和信息隐蔽

- 公有接口与私有实现的分离

  类的功能实现：通过成员函数对数据成员进行操作

  ■公用接口（public interface）：公用成员函数

  ■私有实现（private implementation）：类的功能实现细节对用户隐蔽

  ■信息隐蔽（information hiding）：公用接口与私有实现的分离

■ C++程序包含3各部分：

类声明头文件（后缀为.h或无后缀），用户使用类库的有效方法和公用接口

类实现文件（后缀为.cpp），包括成员函数的定义

类的使用文件（后缀为.cpp），即主文件

```cpp
// main.cpp 主文件
#include <iostream>
#include "student.h"
using namespace std;
int main()
{
  Student stud;
  stud.setdata();
  stud.display();
  return 0;
}
```

```cpp
// student.h 类声明头文件
class Student
{
  private:
    int num;
    string name;
    char sex;
  public:
    void display();
};
```

```cpp
// student.cpp 类实现文件
#include <iostream>
#include "student.h"
using namespace std;
void Student::display()
{
  cout<<"num:"<<num<<endl;
  cout<<"name:"<<name<<endl;
  cout<<"sex:"<<sex<<endl;
}
```

第三章 类和对象的使用

0. 对象的初始化

- 全部都是公有成员 → 可以对象定义时大括号初始化。

```cpp
class Time {
public:
    int hour;
    int minute;
    int sec;
};

Time t1 = {14, 56, 30};  // 合法，逐个赋值
```

用构造函数实现数据成员的初始化

- 建立对象时自动执行，不需要用户调用
- 函数名与类名必须相同
- 不具有类型，无返回值
- 不需要也不能被用户调用

C++ 构造函数(Constructor)写法总结

1. 默认构造函数

- 没有参数，用来做基础初始化。

```
class Time {
public:
    Time() { hour = 0; minute = 0; sec = 0; }
private:
    int hour, minute, sec;
};
```

1. 类外定义构造函数

- 在类中声明，在类外实现。

```
class Time {
public:
    Time();
private:
    int hour, minute, sec;
};

Time::Time() { hour = 0; minute = 0; sec = 0; }
```

1. 带参数的构造函数

- 通过传参为不同对象赋不同初始值。

```
class Box {
public:
    Box(int h, int w, int l) { height=h; width=w; length=l; }
private:
    int height, width, length;
};

Box b1(10, 20, 30);
```

1. 构造函数重载

- 同一个类里可以有多个构造函数，根据参数不同自动匹配。

```
class Box {
public:
    Box() { height=10; width=10; length=10; }
    Box(int h, int w, int l) { height=h; width=w; length=l; }
private:
    int height, width, length;
};
```

1. 使用初始化列表

- 效率更高，直接初始化成员。
- 常量成员(const)和引用成员(&)只能使用初始化列表

```
class Box {
public:
    Box(int h, int w, int l) : height(h), width(w), length(l) {}
private:
    int height, width, length;
};
```

1. 构造函数使用默认参数

- 给构造函数的参数设默认值，可以少传实参。

```cpp
class Box {
public:
    Box(int h=10, int w=10, int l=10) : height(h), width(w), length(l) {}
private:
    int height, width, length;
};

Box b1;             // (10,10,10)
Box b2(15);         // (15,10,10)
Box b3(15,30);      // (15,30,10)
Box b4(15,30,20);   // (15,30,20)
```

- 总结
  - **默认构造函数**：无参，给固定初始值。
  - **类外定义**：声明在类里，实现写在外面。
  - **带参构造函数**：传参初始化对象。
  - **构造函数重载**：多个构造函数，匹配不同参数。
  - **初始化列表**：更高效的写法。
  - **默认参数**：构造函数可带默认值，调用时可省略部分实参。

析构函数(Deconstructor)

- 用于释放对象，不需要用户调用，释放对象时自动执行
- 特殊的成员函数，函数定义 `~类名(){...};`
- 顺序:先构造的后析构，后构造的先析构

1. 对象数组

可以这样建立构造函数并在建立对象数组时调用：

```cpp
Student :: Student ( int=1001,int=18,int=60 );
Student stud[n]={
Student ( 实参1,实参2,实参3 ),
… …
Student ( 实参1,实参2,实参3 ),
};
```

2. 对象指针

指向对象的指针

```
类名 *对象指针名; //定义格式
Time *pt; //定义pt是指向Time类对象的指针;
Time t1; //定义Time类对象t1
pt = &t1; //将对象t1的地址赋给pt
//用指针变量访问对象的成员
(*pt).hour
pt→hour
(*pt).show_time()
pt→show_time()
```

指向数据成员的指针

```
Time t1;
int *p1; //定义一个指向整型数据的指针变量
p1 = & t1.hour;  /计算公有数据成员的地址
```

指向函数的指针(函数指针)

- 指向普通函数的指针变量: `数据类型 (*指针变量名)(形参表);`

```
int print(int a); //已有的函数
int (*pt)(int a); //定义指向函数的指针
pt = &print; //赋值函数的地址
pt = print; //与上一行是等效的
//在之后调用函数，以下两种是等效的:
pt(a);
(*pt)(a);
print(a);
```

💡

在C/C++中， `函数名` 和 `&函数名` 完全等价，原因是函数名（如 `print` ）本身就会 **退化为函数地址**

💡

`*` 在C/C++中的两个角色：

1. 定义时：声明指针 `int* p;`
2. 使用时：解引用，访问指针指向的对象 `cout << *p;`

注意： `*` 实际上和变量名绑定，而不是和数据类型绑定，空格位置不影响含义，例如：

```
int* a, b;    // a 是 int*，但 b 是 int，不是指针!
int *a, *b;   // a 和 b 都是 int* （更清晰）
int *pt(int);    // 传统 C 风格
int* pt(int);    // 有人觉得更直观，返回值是 int*
int * pt(int);   // 更随意的写法
```

指向成员函数的指针

- 定义： `返回类型 (类名::*指针名)(参数列表);`
- 赋值： `指针名 = &类名::成员函数名;` 或 `指针名 = 类名::成员函数名;`
- 调用： `(对象.*指针名)(实参);`

```
void (Time::*p3)();
p3=&Time::get_time;
(t1.*p3)();
```

- 注意:写成员函数指针的时候以上任何一个*或&都不能省去

3. 共用数据的保护

- `const`
  - `const 类型名 变量名` 与 `类型名 const 变量名` 完全等价
  - 对指针小心写法：

```
//指向常数的指针
int const *p;
const int *p;
//常指针（指针的指向地址不能改变），指向的类型是int而不是int const
int * const p;
//指向常数的常指针
const int * const p;
```

- 常量引用:不能通过引用修改它指向的值，但原始变量自己是能修改的

```
int a1 = 10;
int &b1 = a1;
int const &b2 = a1; //常量引用
b1++; //正确
//b2++; //错误
```

- 常对象

  `const 类名 对象名(可选的实参表);`

  - 常对象下的普通成员函数不可调用，只能调用常成员函数

```
const Time t1(10,15,36); //使用默认构造函数定义了常对象
t1.get_time(); //错误，不能调用
//但如果定义成员函数时声明了const，就可以调用
void get_time() const
```

  - 常对象中要修改某个数据成员，只能修改声明为了可变的数据成员: `mutable 类型 数据成员`

- 常数据成员

  `const 类型 数据成员名`

  只能通过构造函数的参数初始化列表进行初始化，不能在构造函数中赋值

- 常成员函数

  `数据类型 函数名 (参数表) const{}`

  - 不能修改数据成员，不能调用非常成员函数
  - 常对象只能调用常成员函数

数据成员	非 const 成员函数	const 成员函数
非 const 的数据成员	可以引用，也可以改变值	可以引用，但不可以改变值
const 数据成员	可以引用，但不可以改变值	可以引用，但不可以改变值
const 对象的数据成员	不允许引用	可以引用，但不可以改变值

- 指向对象的常指针

  `类名 * const 指针变量名 = 对象地址`

  必须在定义时对其初始化（所有const变量都是这样）

- 指向常变量的指针变量

  `const 类型名 *指针变量名;`

- 函数传递出参数三种方法

形式	是否拷贝对象	能否修改原对象	效率	常见用途
`void f(Box b)`	✅ 会拷贝	❌ 不能修改	低（对象大时更低）	小对象传值、测试拷贝构造
`void f(Box& b)`	❌ 不拷贝	✅ 可以修改	高	需要修改调用者的对象
`void f(const Box& b)`	❌ 不拷贝	❌ 不能修改	高	最常用，传入只读大对象

- 小结

形式	含义
`Time const t1;` 或 `const Time t1;`	t1 是常对象，其值在任何情况下都不能改变
`void Time::fun() const`	fun 是 Time 类中的常成员函数，可以引用，但不能修改本类中的数据成员
`Time* const p;`	p 是指向 Time 对象的常指针，p 的值（即 p 的指向）不能改变
`const Time* p;`	p 是指向 Time 类常对象的指针，其指向的类对象的值不能通过指针来改变
`Time& t1 = t;`	t1 是 Time 类对象 t 的引用，t 和 t1 指向同一段内存空间

4.对象

- `new 类名(...)` → 在堆上动态分配对象，返回指针。
- `对象指针→成员` → 访问对象成员。
- `delete 指针` → 释放对象。

```
Box *pt = new Box(12,15,18);
cout << pt→height;
cout << (*pt).height;
delete pt;
```

5. 对象的赋值和复制

- 对象的赋值 `对象1 = 对象2`

- 对象的复制：用已有对象去新建另一个对象
  - `Box box2(box1)`
  - `Box box3 = box1`
    - 注意：`box3 = box1` 是赋值，`Box box3 = box1` 是复制
  - 等价写法，都是利用**复制构造函数**

```
Box::Box(const Box &b) {
    height = b.height;
    width  = b.width;
    length = b.length;
}
```

    参数必须是类的引用，且常加const修饰。若不写，编译器会自动生成一个“默认复制构造函数”，进行逐个成员复制

  - 在以下情况调用复制构造函数：
    - 用一个对象创建另一个对象
    - 函数的参数时对象（按值传递对象）
    - 函数返回一个对象

6. 不同对象间实现数据共享

静态数据成员

```
class Box
{public:
    static int height;
};
int Box::height = 10;

Box a;
cout << a.height; // 通过对象访问
cout << Box::height; // 通过类名访问
```

特点总结

1. **只能在类外初始化**

   静态成员属于整个类，而不是某个对象，所以不能在构造函数里初始化。

2. 全类共享

不管多少个对象，静态成员只有一份。改变一个对象的静态成员，所有对象都会“看到”变化。

3. 内存位置

静态成员存放在**静态存储区**（类似全局变量），而不是对象的内存里。

所以对象体积不会因为 `static` 成员变大。

4. 访问方式

- `对象名.静态成员` ✅（可行）
- `类名::静态成员` ✅（推荐，更直观）

5. 作用域

静态成员和类有相同的作用域，但生命周期是整个程序运行期。

静态成员函数

定义格式： `static 类型 成员函数(形参表){...}`

调用格式： `类名::成员函数(实参表)` `对象名.成员函数(实参表)`

特性	静态成员函数	普通成员函数
所有对象共享	Yes	Yes
是否隐含 `this` 指针	No	Yes
能否访问普通成员	No	Yes
能否访问静态成员	Yes	Yes
是否能通过类名直接调用	Yes	No
是否能通过对象名调用	Yes	Yes

7. 允许访问私有数据的“朋友”——友元（friend）

1. 友元的作用

- 允许访问类的 **私有成员** 和 **保护成员**。
- 用途：突破类的封装，方便特殊场景下直接访问数据。
- 缺点：破坏封装性，**建议少用**。

2. 友元的种类

1. 友元函数

- 普通函数作为友元

```
class Time {
    friend void display(Time &);
private:
    int hour, minute, sec;
};
void display(Time &t) { cout << t.hour; }

```

- 另一个类的成员函数作为友元

```
class B {
    void disp(A temp);  // 作为友元
};
class A {
    friend void B::disp(A);
private:
    int x;
};

```

2. 友元类

- 整个类作为友元

```
class A {
    friend class B;
private:
    int x;
};
class B {
    void fun(A a) { cout << a.x; }
};

```

---

3. 特点

- **单向性**：A是B的友元 ≠ B是A的友元。
- **不可传递**：A是B的友元，B是C的友元 ≠ A是C的友元。
- 一般不把整个类声明为友元类，而是只让 **需要的函数** 成为友元。

8. 类模板

- 定义格式：

```
template <class 类型参数名>
class 类模板名
{...}
```

- 举例：

```
template <class numtype>
class Compare
{private:
    numtype x,y;
    public:
    compare(numtype a,numtype b)
    {x = a y = b;}
    numtype max();
    }
numtype Compare<numtype>::max()
    {return (x>y)?x:y;}
```

- 使用类模板

```
类模板名 <实际类型名> 对象名;
类模板名 <实际类型名> 对象名(实参表);

Compare<int> cmp1(3,7);
```

在编译时，会用int取代numtype，这时Compare相当于Compare_int类

第四章 运算符重载

0. 为什么要对运算符重载

- 重载(overloading)：一名多用
- 运算符重载(operator overloading):用一个运算符表示不同功能的运算
- 例： `<<` 左移运算符， `cout<<` 流插入运算符

1. 对运算符重载的方法

- 格式: `函数类型 operator 运算符名称(形参表) {重载处理}`
- 两种形式: 重载为类成员函数；重载为友元函数
- 例子：复数相加（类成员函数）

```
class Complex
{
    ... \\ 定义实部和虚部
    Complex operator+(Complex &x2);
}

Complex Complex::operator+(complex &c2)
{
    ...\\ 实现相加
}
int main()
{
    ... \\定义c1,c2,c3
    c3 = c1+c2;
}
```

2. 运算符重载的规则

1. C++只允许对已有的部分运算符实施重载，不能重载的运算符有: `::` `.` `sizeof` `.*` `?"`
2. 重载不改变操作数的个数、优先级和结合性
3. 运算符重载函数不能带默认值参数，必须与自定义的对象联合使用，参数中至少要有类对象或类对象引用
4. 运算符重载函数可以作为类成员函数、友元函数、普通函数

3. 运算符重载函数作为类成员函数和友元函数

运算符重载为友元函数

两个复数相加

```
class Complex{
 public:
    Complex(){real=0;image=0;}
    Complex(double r, double i) : real(r), image(i) {}
    friend Complex operator+(Complex &c1,Complex &c2);
 private:
    double real; double image;
}
Complex operator+(Complex &c1, Complex &c2{
    return Complex(c1.real+c2.real,c1.image+c2.image);
};
```

解释：在编译时将 `c1+c2` 解释为 `operator+(c1,c2)`

顺序、交换律

- 友元函数不要求第一个操作数是对象，但是顺序要和函数定义一致
- 成员函数第一个操作数一定是对象，并且函数定义中不会写
- 实现对象与非对象相加的加法交换律的方法：
  - 成员函数+友元函数(右操作数为对象)
  - 友元函数(左操作数为对象)+友元函数(右操作数为对象)

4. 重载双目运算符

- 双目：两个操作数（`+ - * / == < >`）。
- 通常用 **友元函数**，两边操作数都能参与。
- 成员函数 → 左操作数必须是类对象。
- 示例：字符串比较 `>, <, ==`。

5. 重载单目运算符

- 单目：一个操作数（`++ -- !`）。

- **前置++**：`Time operator++()` → 返回自增后对象。
- **后置++**：`Time operator++(int)` → 返回自增前对象（哑元区分）。

6. 重载流插入/提取运算符

- `cin` / `cout`：必须定义为 **友元函数**。
- 格式：

```
1  friend ostream& operator<<(ostream&, 类&);
2  friend istream& operator>>(istream&, 类&);
```

- 返回引用 → 支持连续输入输出。

7. 归纳

- 作用：让程序更直观、简洁。
- **规则**：
  - 单目/复合运算符 → 一般成员函数。
  - 双目运算符 → 一般友元函数。
  - `= [] () →` → 必须成员函数。
  - `<< >>` → 必须友元函数。
- 引用：参数避免复制，返回值可作左值。

8. 不同类型数据间的转换

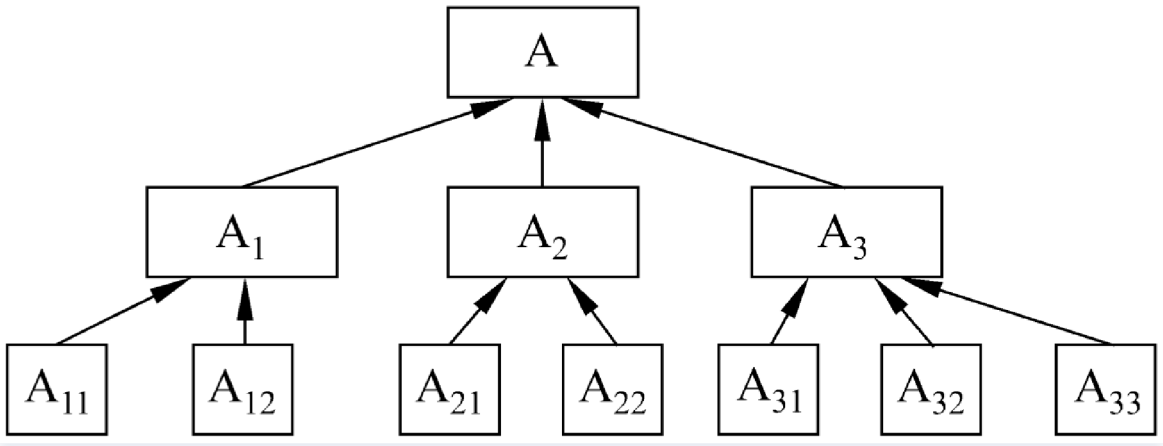
- 标准类型数据转换：`类型名(数据)` `(类型名)数据`
- 其他类型数据转换为类对象
  - `类名(指定类型的数据)`
  - 转换构造函数：只有一个形参
  - `Complex(double r){real = r; image = 0;}`
- 类对象转换为其他类型数据
  - 类型转换函数 `operator 类型名 () {转换语句}`

```cpp
operator double() {return real;}
double d;
d = 2.5+c1; //c1转成double再相加
c3 = c1+c2; //两个Complex相加
d = c1+c2; //两给Complex相加再转成double
```

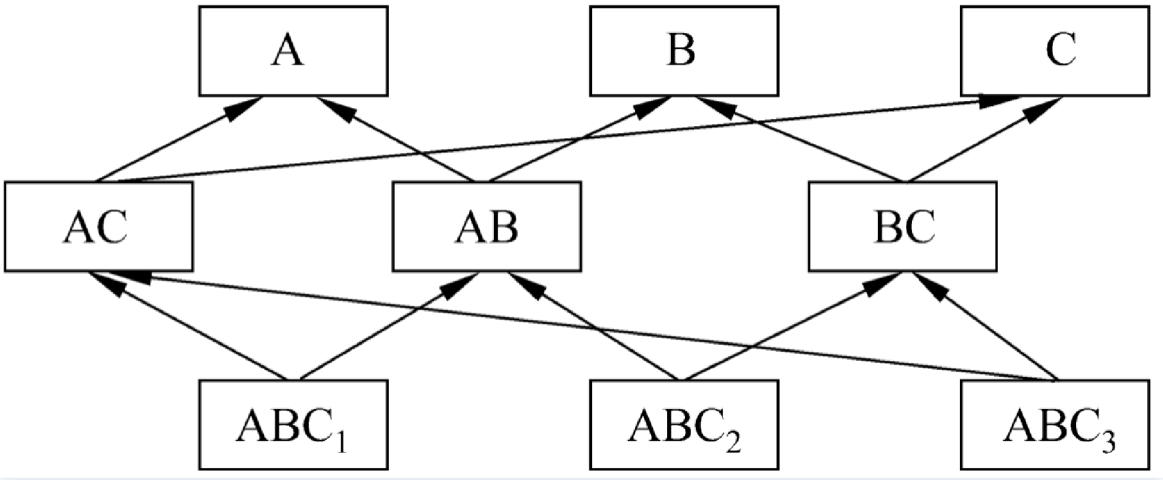
第五章 类的继承

0. 继承与派生

- 基类(base class)或父类
- 派生类(derived class)或子类
- 派生类继承了基类的所有数据成员和成员函数
- 每个派生类可以作为基类再派生出新的子类



- 可以多重继承



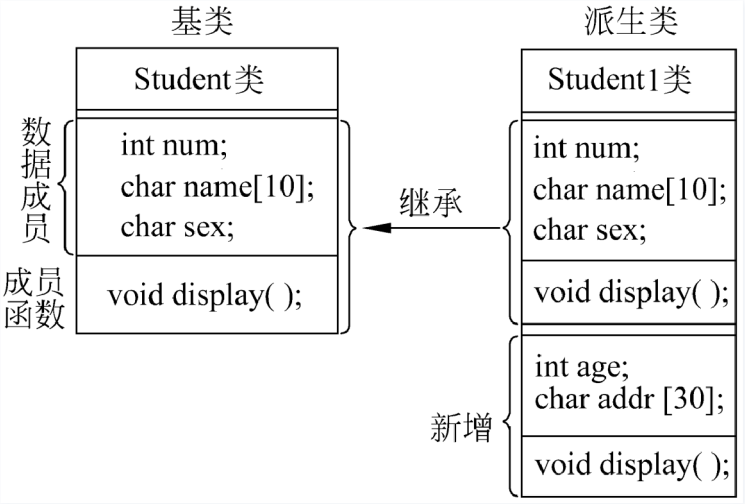
1. 派生类的声明方式

```
class 派生类名:[继承方式] 基类名
{派生类新增成员声明};

class Student1 : public Student
{
    private:
        int age;
    public:
        void display();
}
```

- 继承方式决定基类成员在派生类中的可见性。包括:public,private,protected
  - **public 继承**：保持原有权限（最常用）。
  - **protected 继承**：对外收缩，基类 public 在子类中变 protected。
  - **private 继承**：对外完全隐藏，基类 public/protected 在子类中都变 private。

2. 派生类的构成



1. 从基类接受成员

- 派生类会继承基类 **除构造函数和析构函数外的所有成员**。

2. 调整从基类接受的成员

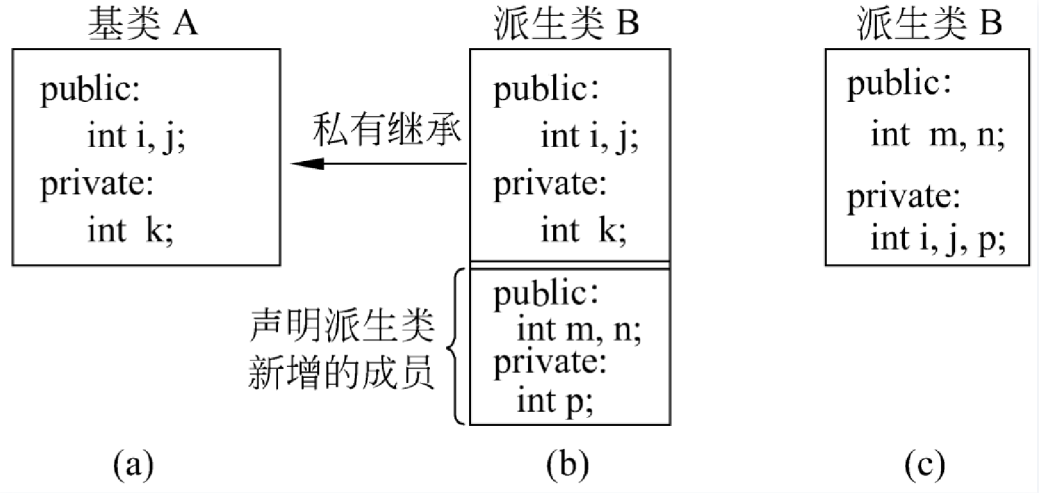
- 派生类中可以声明一个与基类同名的成员 → **屏蔽基类的同名成员**。
- 注意：屏蔽要求 **函数名相同且参数列表也相同**。
- 含义：用派生类成员 **替代/覆盖** 基类成员。

3. 在声明派生类时增加的成员

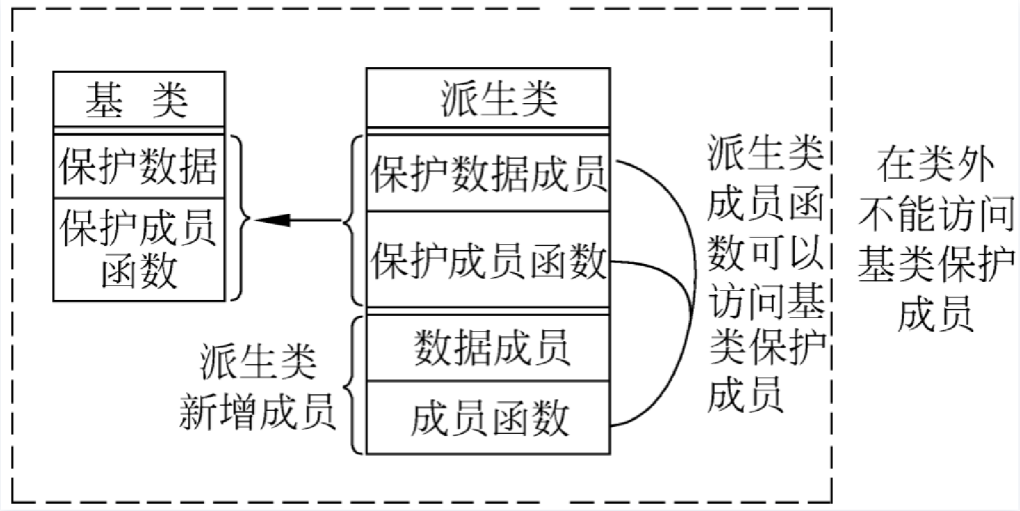
- 体现派生类对基类功能的 **扩充**。
- 在派生类中可以定义自己的 **构造函数、析构函数、普通成员函数** 等。

3. 派生类成员的访问属性

- 私有继承 private inheritance



- 保护继承 protected inheritance



基类成员权限	公有继承	保护继承	私有继承
`public`	public	protected	private
`protected`	protected	protected	private
`private`	❌不可见	❌不可见	❌不可见

派生类中访问属性	在派生类中	在派生类外	在下层公用派生类中
公有	可以	可以	可以
保护	可以	不可以	可以
私有	可以	不可以	不可以
不可访问	不可以	不可以	不可以

4. 派生类的构造函数与析构函数

- 基本思路：执行派生类的构造函数时，调用基类的构造函数

```
// 一般形式
派生类名(基类所需的形参,本类成员所需的形参):基类名(基类实参表)
{
    本类成员初始化赋值语句;
}
// 写在类外时，第一个派生类名表示这是哪一个类的
派生类名::派生类名(基类所需的形参,本类成员所需的形参):基类名(基类实参表)
{
    本类成员初始化赋值语句;
}
```

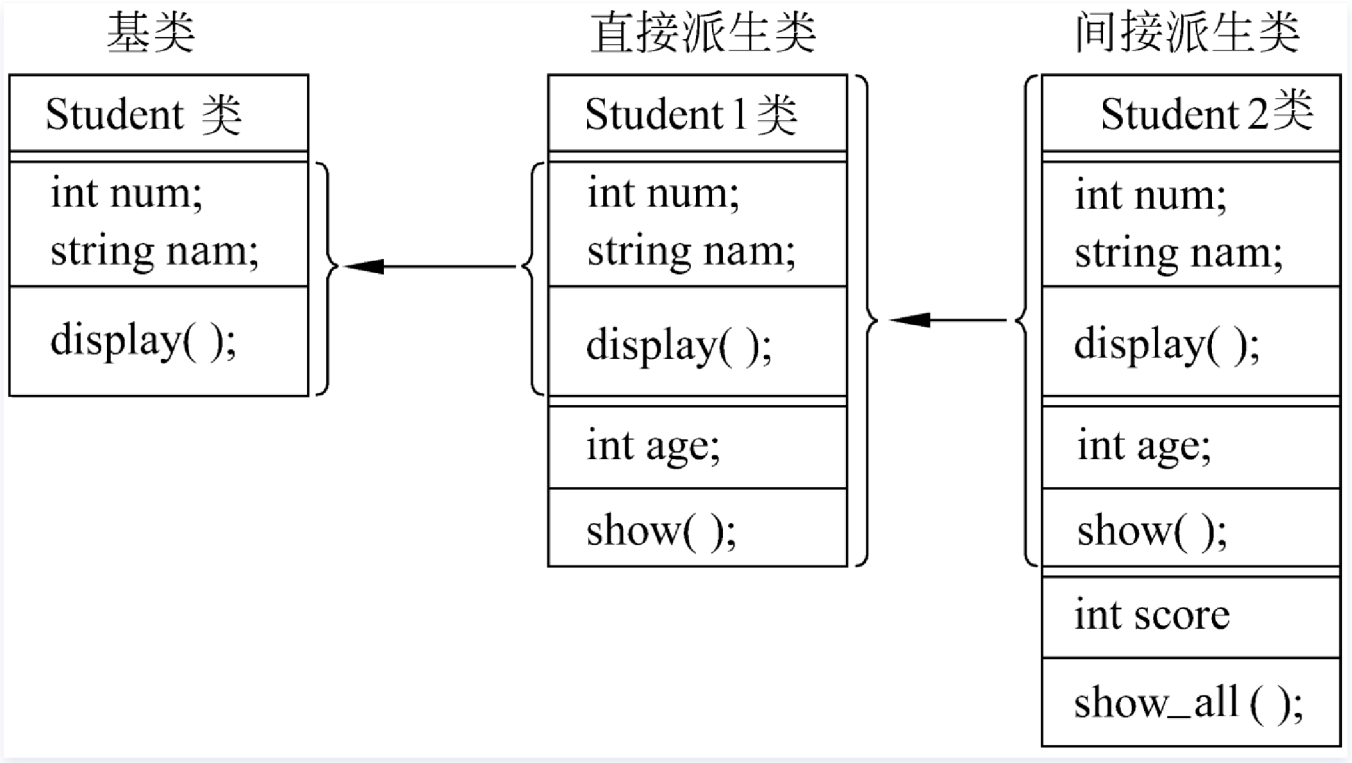
执行顺序：

- 构造函数；基类、派生类
- 析构函数：派生类、基类
- 有子对象的派生类的构造函数

```
派生类名::派生类名(基类所需的形参,本类成员所需的形参):基类名(基类实参表),子对象名(参数表)
{
    本类成员初始化赋值语句;
}
```

执行顺序：基类、子对象、派生类

- 多层派生时的构造函数



写法；只需要调用直接基类的构造函数，不需要再管它的间接基类

```
// 基类的构造函数
Student(int n, string name);
// 派生类Student1的构造函数
Student1(int n, string name, int a):Student(n,name);
// 派生类 Student2 的构造函数
Student2(int n, string name, int a, int s):Student1(n,name,a);
```

5. 多重继承

多重继承的定义与声明

一个派生类同时继承多个基类

```
class D: public A, private B, protected C
{ D类新增的成员声明 }
```

多重继承派生类的构造函数

```
派生类构造函数名(总参数表): 基类1构造函数(参数表),
                                基类2构造函数(参数表),
                                基类3构造函数(参数表)
{ 派生类新增成员初始化语句 }
```

调用顺序：与构造函数中写的先后顺序无关，按照声明派生类时基类的出现顺序

多重继承引起的二义性问题

- 同名隐藏规则：当派生类与基类中有相同成员时，若未强行指名，则通过派生类对象使用的是派生类中的同名成员
- 如要通过派生类对象访问基类中被覆盖的同名成员，应使用基类名限定。

```
class A
{public:
  void f();
};
class B
{public:
  void f();
  void g();
};
```

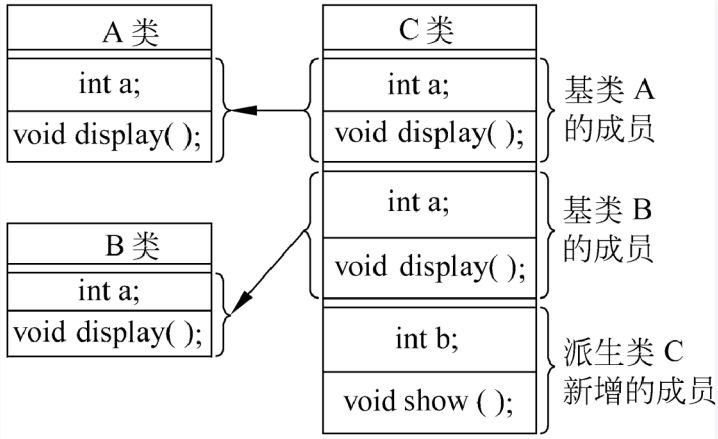
```
class C : public A, public B
{public:
  void g();
  void h();
};

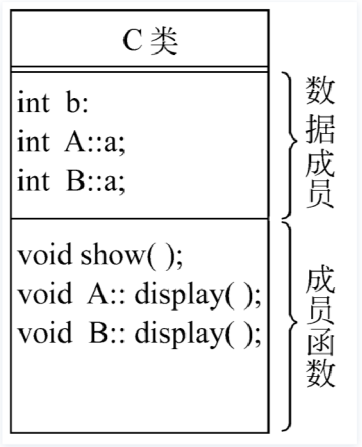
C c1;
c1.f(); // 有二义
c1.g(); // 无二义，同名覆盖
```

```
//解决方法一：
c1.A::f();
c1.B::f();

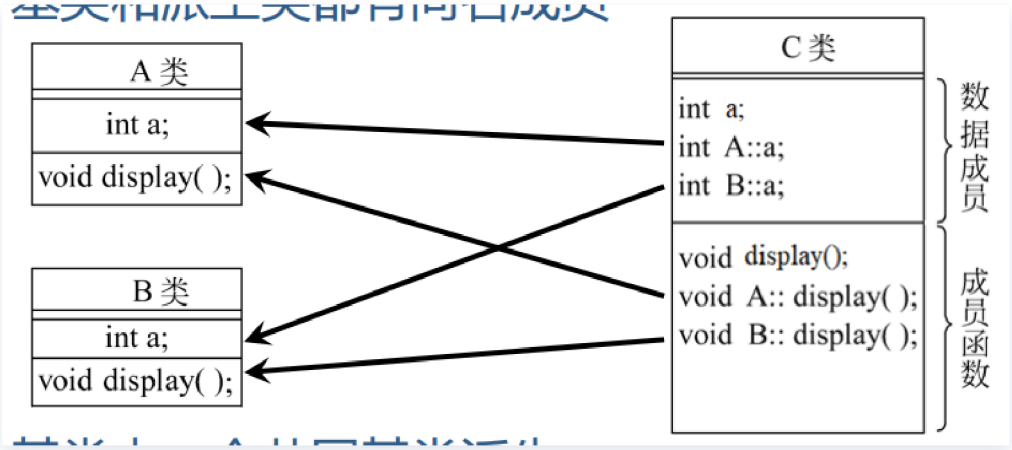
//解决方法二：
同名覆盖
```

1. 两个基类有同名成员

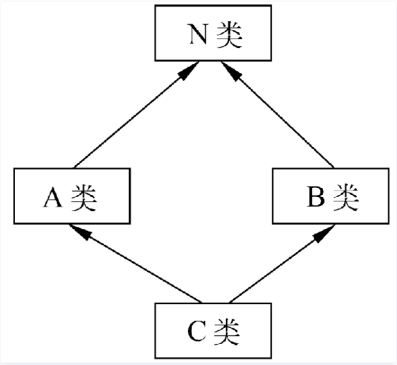


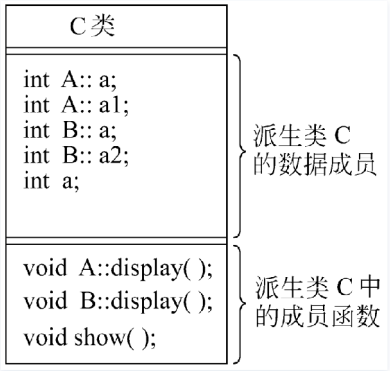


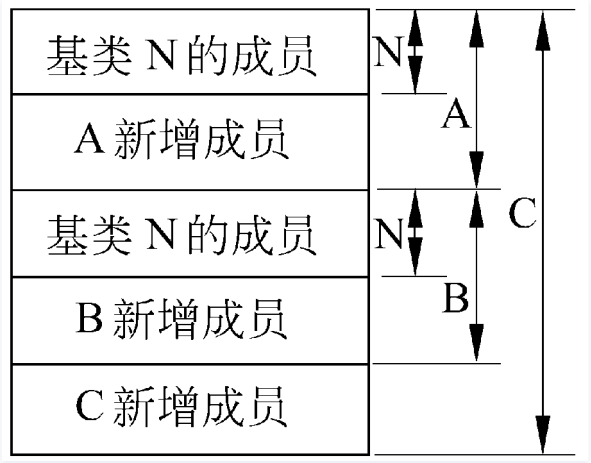
1. 两个基类和派生类都有同名成员



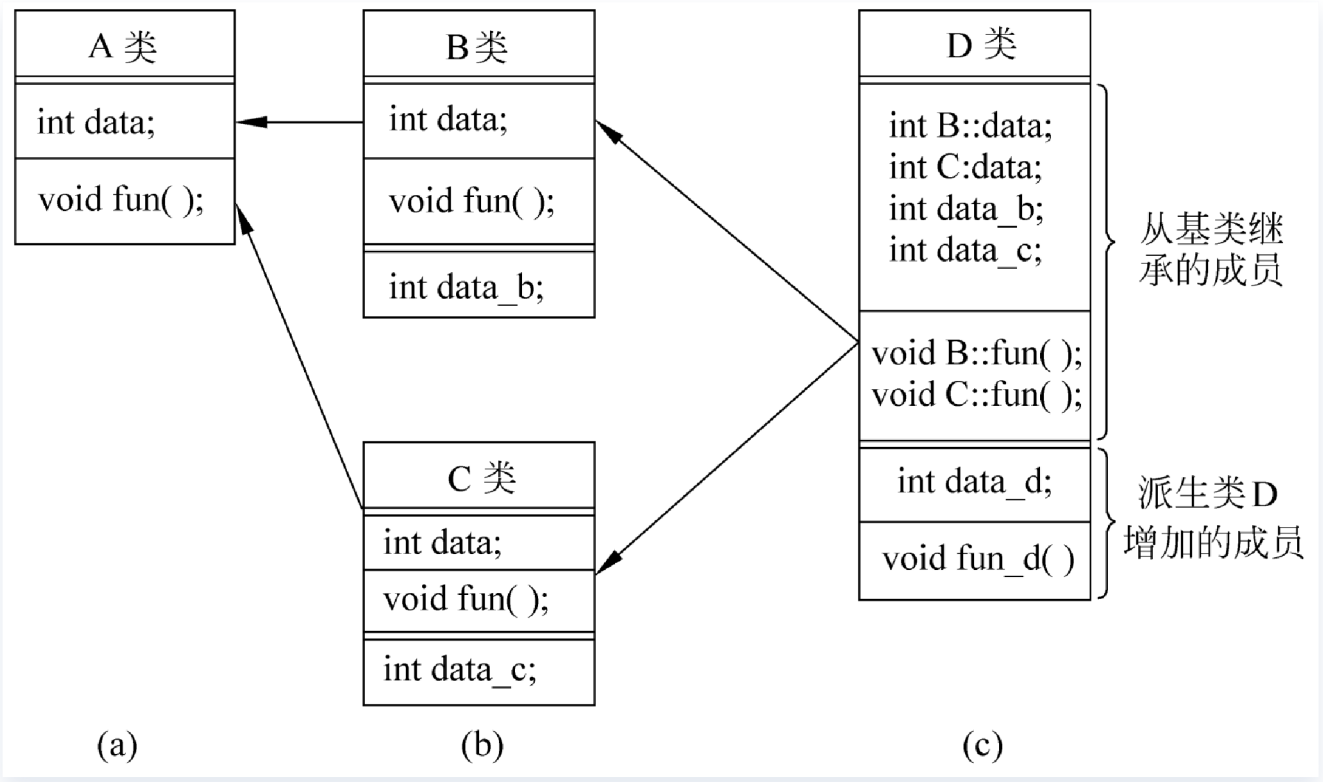
1. 两个基类由一个共同基类派生

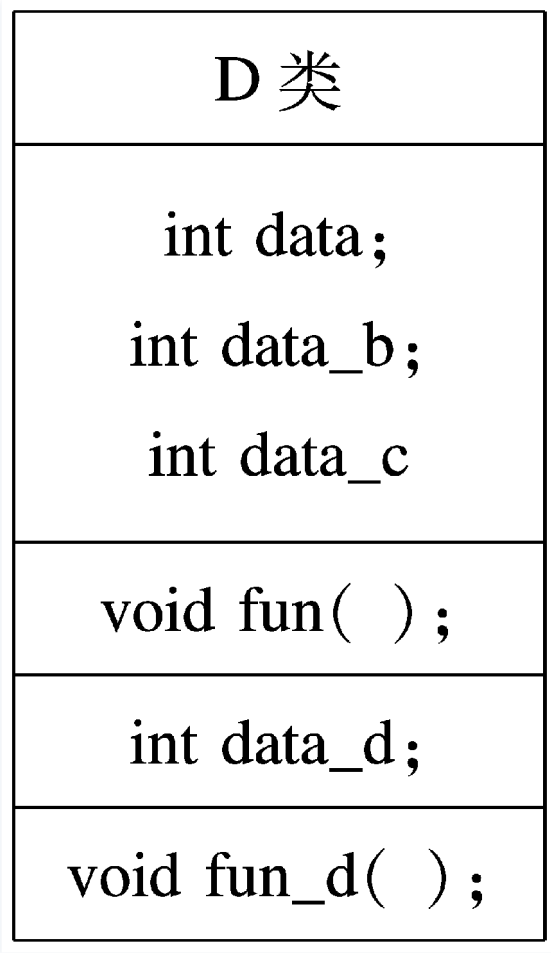






在继承共同基类时减少数据冗余——虚基类





- 多重继承时，如果多个派生类继承同一个基类，会出现重复的基类成员，造成数据冗余和二义性。用 `virtual` 声明基类为虚基类，可以使最底层派生类只保留一份该基类成员。
- 声明格式：

```
class 派生类名 : virtual 继承方式 基类名

```

- 示例（虚基类解决菱形继承问题）：

```
class A {
public:
    int data;
    void show() { cout << "A::data = " << data << endl; }
};

class B : virtual public A {
public:
    B() { data = 10; }
};

```

```
class C : virtual public A {
public:
    C() { data = 20; }
};

class D : public B, public C {
public:
    void print() { cout << "D::data = " << data << endl; }
};

int main() {
    D d;
    d.print();  // 输出 D::data = 20，只保留一份 A 的成员
    return 0;
}

```

- 如果不使用虚继承，类 D 会同时拥有两份 A 的成员，`d.B::data` 和 `d.C::data` 会互不相同。
- 虚基类构造函数的调用由最底层派生类负责，中间的派生类调用会被忽略。

```
class A {
public:
    A(int n) { cout << "A 构造 " << n << endl; }
};

class B : virtual public A {
public:
    B(int n) : A(n) { cout << "B 构造" << endl; }
};

class C : virtual public A {
public:
    C(int n) : A(n) { cout << "C 构造" << endl; }
};

class D : public B, public C {
public:
    D(int n) : A(n), B(n), C(n) { cout << "D 构造" << endl; }
};

int main() {
    D d(5);
    return 0;
}
```

```
25
```

输出顺序表明：`A` 的构造函数只执行一次，说明虚基类只初始化一次。

- 构造顺序：先虚基类，再直接基类，最后派生类自身。
- 虚基类的好处是避免重复成员，提高数据一致性，常用于复杂继承结构中。

6. 基类与派生类的转换

- 公有继承下，派生类对象是一种特殊的基类对象，可以与基类之间进行类型兼容。
- 派生类对象可以赋值给基类对象，但此时会发生对象切割，只保留基类成员。
- 派生类对象可以通过基类的指针或引用访问，但只能访问基类部分，无法访问派生类新增成员。
- 当函数参数是基类引用或指针时，可以传入派生类对象，这种机制构成了多态的基础。
- 示例中 `Student` 为基类，`Graduate` 派生自它并增加成员 `wage`。
- 当基类指针 `pt` 指向派生类对象并调用 `display()` 时，只执行基类版本的函数，无法输出 `wage`。
- 若想根据对象的真实类型调用不同版本函数，应将基类的 `display()` 声明为 `virtual`，这样会根据实际对象类型动态绑定，实现多态。

第六章 多态性

1. 什么是多态性

同一个函数调用，在不同类型的对象上产生不同的结果

2. 利用虚函数实现动态多态性

- 在基类中用关键字 `virtual` 声明函数，使其成为虚函数。
- 当用**基类指针或引用**调用该函数时，程序会在**运行时**决定调用哪个版本（基类的或派生类的），这称为**动态绑定**。

```
1  class Animal {
2  public:
3      virtual void speak() { cout << "Animal speaks" << endl; }
4  };
5
6  class Dog : public Animal {
7  public:
```

```
    void speak() override { cout << "Dog barks" << endl; }
};

int main() {
    Animal *p = new Dog;
    p->speak();    // 输出 "Dog barks"，因为 speak() 是虚函数
}

```

```
class Student {
    ....
    public: virtual void display();
    };
class Graduate : public Student {
    ...
    public: virtual void display();
    // 这里不写virutal，也依然是虚函数，也就是可以写成：
    // void display();
    };
int main() {
    Student stud1;
    Graduate grad1;
    Student* pt1 = &stud1;
    Student* pt2 = &grad1;
    pt1->display(); // 会调用Student类中的display();
    pt2->display(); // 会调用Graduate类中的display();
    }
```

虚函数的使用方法：

- 在基类中用关键字 `virtual` 声明函数，使其成为虚函数，例如

```
class Base {
public:
    virtual void show();
};
```

- 在派生类中可以重写（override）这个虚函数，函数名、参数列表和返回类型要与基类完全一致：

```cpp
class Derived : public Base {
public:
    void show() override;  // override 可写可不写，写上可读性更好
};
```

- 使用**基类指针或引用**指向**派生类对象**时，调用虚函数会根据对象的实际类型在运行时确定（动态绑定）：

```cpp
Base *p = new Derived;
p->show();  // 调用 Derived::show()
```

- 如果去掉 `virtual`，则是静态绑定，编译时就确定调用基类版本。
- 通常在基类中需要有一个虚析构函数，以保证通过基类指针删除派生类对象时正确析构：

```cpp
virtual ~Base() {}
```

- 虚函数不能在构造函数或析构函数中实现多态调用，也不能是静态成员函数。

派生类中写不写 `virtual`，在功能上通常没有区别。

在 C++ 中，**虚函数的特性是会被继承的**。

- 如果基类中的函数被声明为 `virtual`，
- 那么在派生类中，即使你**不再写** `virtual`，该函数依然是虚函数。
- 虚函数看对象类型，非虚函数看指针类型。
  - 虚函数 → 动态绑定 → 调用实际对象对应的版本。
  - 非虚函数 → 静态绑定 → 调用指针（或引用）声明类型对应的版本。

```cpp
class Base {
public:
    virtual void fun();
    virtual void gun();
    virtual void sun();
    void run();
};
class Derived : public Base {
public:
    void fun();
    virtual void gun();
    void run();
```

```cpp
    virtual void mun();
};
int main() {
    Base bobj;
    Derived dobj;
    Base *bp = &dobj;
    Derived *dp = &dobj;

    // 直接访问基类
    // 全部访问基类中的成员
    bobj.fun();
    bobj.gun();
    bobj.sun();
    bobj.run();

    // 直接访问派生类
    // 优先访问派生类中重写的成员（同名隐藏）
    dobj.fun();  // Derived::sun();
    dobj.gun();  // Derived::gun();
    dobj.sun();  // Base::sun;
    dobj.run();  // Derived::run();
    dobj.mun();  // Derived::mun();

    // 用基类指针访问派生类
    bp->fun();  // Derived::fun();  虚函数，动态绑定：根据对象实际类型绑定
    bp->gun();  // Derived::fun();  虚函数，动态绑定：根据对象实际类型绑定
    bp->sun();  // Base::sun();
    bp->run();  // Base::run();  普通成员函数，同名覆盖  静态绑定：只看指针的静态类型Base*

    bp->mun();  // 报错

}
```

- 虚析构函数
  - 用基类指针访问派生类的普通析构函数：调用基类的析构函数，只能释放基类部分的内存
  - 用基类指针访问派生类的虚析构函数：调用派生类的析构函数，可以释放全部内存

3. 纯虚函数与抽象类

- 没有函数体的纯虚函数
  - 只声明，不定义
  - 格式：在普通虚函数声明末尾加上 `=0` 即表示它是虚函数

- 目的：提供外部接口

- 抽象类
  - 只要存在纯虚函数，就是抽象类
  - **不能实例化**，不能创建对象
  - 可以作为基类，由子类实现其中的纯虚函数
- 示例

```
class Shape {
public:
    virtual void draw() = 0;  // 纯虚函数
    virtual double area() = 0;
};
class Circle : public Shape {
public:
    void draw() override {
        cout << "Draw Circle" << endl;
    }
    double area() override {
        return 3.14 * r * r;
    }
private:
    double r;
};

// 使用
Shape *p = new Circle();
p->draw();     // 调用 Circle::draw()
cout << p->area();
delete p;
```

总结：与”虚“有关的概念解析

名称	所属范畴	关键字	是否涉及继承	主要作用	是否能实例化
虚函数（virtual function）	函数机制	`virtual`	✅ 是	实现多态（运行时动态绑定）	✅ 可以
纯虚函数（pure virtual function）	函数机制	`virtual ... = 0`	✅ 是	只定义接口、不实现，让子类必须重写	❌ 不行（定义它的类不能实例化）
抽象类（abstract class）	类机制	含纯虚函数	✅ 是	充当接口规范，强制派生类实现行为	❌ 不行

名称	所属范畴	关键字	是否涉及继承	主要作用	是否能实例化
虚基类（virtual base class）	继承机制	`virtual public`	✅ 多继承中	解决菱形继承中基类重复问题	✅ 可以

第7章 输入输出流

0. C++的输入和输出

C++的输入输出含义

- **(1) 标准设备的输入和输出 (标准I/O)**
  - 标准输入: 键盘输入
  - 标准输出: 显示器输出
- **(2) 以外存为对象进行输入和输出 (文件I/O)**
  - 指的是文件输入输出
- **(3) 对内存中制定的空间进行输入和输出 (串I/O)**
  - 字符串输入输出: 指定一个字符数组作为储存空间

C++ I/O 对 C 的发展

- C语言的 `printf` 和 `scanf` 存在类型安全问题，例如 `printf("%d\n", f);`（用 `%d` 输出浮点数）或 `scanf("%d", i);` (漏写 `&`) 可能会导致严重问题。
- C++的输入输出进行了严格的类型检查，类型不正确的无法通过编译，因此是**类型安全 (type safe)**的。
- 通过**重载**方式，C++的I/O具有**可扩展性 (extendibility)**，允许标准类型和用户声明类型都采用相同方法处理。

C++的输入输出流 (input output stream)

- (1) 是指若干字节组成的字节序列按顺序从一个对象传送到另一个对象。
- (2) 缓冲区中（输出缓冲区、输入缓冲区）的数据就是流。
- (3) C++的I/O库（流类库）中的类称为**流类 (stream class)**。
- (4) 对应的实例就是**流对象 (stream object)**。例如 `cout`、`cin` 就是 `iostream` 类的对象。
- (5) C++的流库有两个基类： `ios` 类、 `streambuf` 类。

- (6) C++的流库有关的头文件：

头文件	作用
`iostream`	输入输出流操作
`fstream`	管理文件的I/O操作
`strstream`	字符流的I/O操作
`stdiostream`	混合使用C和C++
`iomanip`	使用格式化I/O操作

- (7) `iostream` 头文件中定义的流对象：

对象	含义	设备	C语言中对应的标准文件
`cin`	标准输入设备	键盘	`stdin`
`cout`	标准输出设备	屏幕	`stdout`
`cerr`	标准错误输出设备(非缓冲方式)	屏幕	`stderr`
`clog`	标准错误输出设备(缓冲方式)	屏幕	`stderr`

1. 标准输出流

cout, cerr和clog流

- (1) `cout` 流在内存中对应开辟了一个缓冲区。
- (2) 向 `cout` 流插入 `endl` 时，会立即输出流中的所有数据，然后插入一个换行符并清空缓冲区。
  - 注意：`\n` 只会换行，不会刷新缓冲区。
- (3) `cerr` 是标准出错信息流，不经过缓冲区直接向显示器输出出错信息。
- (4) `clog` 是标准出错流，它把出错信息存放在缓冲区，当缓冲区满或遇到 `endl` 时才向显示器输出。

标准类型数据的格式输出

1. 使用控制符控制输出格式

- 需要包含头文件 `<iomanip>` 。
- **常用控制符**：

控制符	作用
`dec`	设置整数的基数为10
`hex`	设置整数的基数为16
`oct`	设置整数的基数为8
`setbase(n)`	设置整数的基数为n (n只能是8, 10, 16)
`setfill(c)`	设置填充字符c
`setprecision(n)`	设置实数的精度为n位。在一般形式下代表有效数字，在 `fixed` 和 `scientific` 形式下代表小数位数。
`setw(n)`	设置字段宽度为n位
`setiosflags(ios::fixed)`	设置浮点数以固定的小数位数显示
`setiosflags(ios::scientific)`	设置浮点数以科学记数法(即指数形式)显示
`setiosflags(ios::left)`	输出数据左对齐
`setiosflags(ios::right)`	输出数据右对齐
`setiosflags(ios::skipws)`	忽略前导的空格
`setiosflags(ios::uppercase)`	科学记数法和十六进制的字母A-F大写
`setiosflags(ios::showpos)`	输出正数时给出“+”号
`resetioflags()`	终止已设置的输出格式状态

1. 用流对象成员函数控制输出格式

- **常用成员函数**：

流成员函数	作用	相同控制符
`precision(n)`	设置实数的精度为n位	`setprecision(n)`
`width(n)`	设置字段宽度为n位	`setw(n)`
`fill(c)`	设置填充字符c	`setfill(c)`
`setf()`	设置输出格式状态	`setiosflags()`
`unsetf()`	终止已设置的输出格式状态	`resetioflags()`

- `setf()` 使用的格式标志：

格式标志	作用
`ios::left`	输出数据在本域宽范围内向左对齐
`ios::right`	输出数据在本域宽范围内向右对齐
`ios::internal`	数值的符号位在域宽内左对齐,数值右对齐
`ios::dec`	设置整数的基数为10
`ios::oct`	设置整数的基数为8
`ios::hex`	设置整数的基数为16
`ios::showbase`	强制输出整数的基数(八进制0, 十六进制0x)
`ios::showpoint`	强制输出浮点数的小点和尾数0
`ios::uppercase`	科学记数法和十六进制字母大写
`ios::showpos`	对正数显示“+”号
`ios::scientific`	浮点数以科学记数法格式输出
`ios::fixed`	浮点数以定点格式(小数形式)输出

用流成员函数put输出字符

- **格式**: `cout.put(字符/数字)` 。
- 如果是字符，直接输出该字符。
- 如果是数字 (int)，则输出其对应的ASCII码字符。
- 例如： `cout.put(71);` 会输出 'G'。
- `put` 函数可以连续调用： `cout.put(71).put(79).put(79);` 会输出 "GOO"。

2. 标准输入流

cin流

1. 是 `istream` 类的对象，从标准输入设备（键盘）读取数据。
2. 流提取运算符 `>>` 在读取数据时，通常会跳过流中的空格、tab键、换行符等空白字符。
3. 当遇到无效字符（与变量数据类型不一致）或文件结束符时， `cin` 流就处于出错状态，后续操作被终止。
4. 当输入流出错时， `cin` 的值是 `false` 。这可以用于循环判断，例如 `while (cin >> grade)` 。

用于字符输入的流成员函数

1. **get函数**

- **格式1:** `cin.get()`
  - 从输入流中提取一个字符，并将其作为函数值返回。
  - 如果遇到文件结束符EOF，返回-1。
  - 示例: `while ((c = cin.get()) != EOF)` 。
- **格式2:** `cin.get(字符变量)`
  - 从输入流中提取一个字符，并赋给该字符变量。
  - 示例: `while (cin.get(c))` 。
- **格式3:** `cin.get(字符指针, n, 终止字符)`
  - 从缓冲区最多顺序提取 `n-1` 个字符，放入字符指针所指的字符数组。
  - 如果在提取过程中遇到终止字符，则提前终止提取。
- **get函数特点**:
  - `cin.get()` 函数族**不忽略**提取的空白字符。
  - 格式3（带三个参数）的 `get` 函数在提取结束后，指针**不会**移到终止字符的下一个字符。

1. **getline函数**

- **格式:** `cin.getline(字符指针, n, 终止字符)` 。
- 功能与带三个参数的 `get` 函数类似，都不忽略空白字符。
- **主要区别**: `cin.getline` 在停止提取时，会把指针移到终止字符**之后**相邻的字节。而 `cin.get` (三参数) 不会。

istream类的其他成员函数

- **(1) eof 函数**
  - 当输入缓冲区的指针遇到文件结束符时， `eof()` 函数值为真 (true)。
  - 从键盘输入文件结束符 (Ctrl+Z)。
- **(2) peek 函数**
  - `c = cin.peek();`
  - 返回指针指向的当前字符，但**只是观测**，指针**并不后移**。
- **(3) putback 函数**
  - 将一个字符返回到流中，插入到当前指针的位置。
  - 示例: `cin.putback(ch[0]);` 。

- **(4) ignore 函数**
  - **格式**: `cin.ignore(n, 终止字符)`。
  - 跳过n个字符，或在遇到终止字符时提前结束。
  - 无参数时，默认跳过1个字符。

3. 对数据文件的操作与文件流

文件的概念

1. 文件是指存储在存储介质（如硬盘）上的数据集合。
2. 根据文件中数据的表示形式，文件分为ASCII文件和二进制文件。
3. **ASCII文件**：也称文本文件，每个字节表示一个字符。
4. **二进制文件**：把内存中的数据按其在内存中的格式存放在磁盘上，也称内存数据的映象文件。
5. 字符信息在两种文件中的形式是一样的。但对于数值数据，两者不一样。例如整数100000，用ASCII码表示时用6个字节，而用二进制形式表示时用4个字节。

文件流类与文件流对象

1. 文件流是以外存文件为输入输出对象的流。
2. C++通过三个类来支持文件操作：

- `ifstream` 类: 支持从磁盘文件输入。
- `ofstream` 类: 支持向磁盘文件输出。
- `fstream` 类: 支持对磁盘文件进行输入和输出。

1. 必须定义一个文件流类对象来对文件进行操作。

- `ifstream infile;`
- `ofstream outfile;`
- `fstream iofile;`

文件的打开与关闭

- (1) 打开文件 (Open): 在文件流对象和磁盘文件之间建立关联。
  - **方式一：使用 `open` 函数**C++

    ```
    ofstream outfile;
    outfile.open("f1.txt", ios::out);
    ```

  - **方式二：定义流对象时指定参数**C++

```
ofstream outfile("f1.txt", ios::out);
```

- (2) 关闭文件 (Close): 使用 `close` 函数。
  - `outfile.close();`
- **文件打开模式 (File Modes)**:

方式	作用
`ios::in`	以输入方式打开文件
`ios::out`	以输出方式打开文件 (默认方式)。如果文件已存在, 清除原有内容。
`ios::app`	以输出方式打开, 写入的数据**添加**在文件末尾
`ios::ate`	打开一个已有的文件, 文件指针指向文件末尾
`ios::trunc`	打开文件, 如果已存在则删除全部数据。 `ios::out` 默认包含此方式。
`ios::binary`	以二进制方式打开。默认为ASCII方式。
`ios::in | ios::out`	

对ASCII文件的操作

- 对ASCII码文件（文本文件）的操作主要有两种方法:
  1. 使用文件流对象的提取 `>>` 和插入 `<<` 运算符。
  2. 使用文件流对象的成员函数 `put` , `get` , `getline` 。

对二进制文件的操作

- 在打开时必须指定 `ios::binary` 模式。
- 主要使用 `read` 和 `write` 成员函数。
- **`write` 函数**
  - **格式**: `输出文件流对象.write(内存指针, 长度);`
  - 示例: `outfile.write((char *)&stud, sizeof(stud));`
- **`read` 函数**
  - **格式**: `输入文件流对象.read(内存指针, 长度);`
  - 示例: `infile.read((char *)stud, sizeof(stud));`

随机访问与文件指针

- 磁盘文件用一个指针表示当前要访问的位置。
- 二进制文件允许程序控制指针移动，以实现随机访问。
- **文件指针相关成员函数**：

成员函数	作用
`tellg()`	返回**输入**文件指针的当前位置
`seekg(pos)`	将**输入**文件中指针移到指定的位置 `pos`
`seekg(offset, ref)`	以参照位置 `ref` 为基础移动 `offset` 字节
`tellp()`	返回**输出**文件指针当前的位置
`seekp(pos)`	将**输出**文件中指针移到指定的位置 `pos`
`seekp(offset, ref)`	以参照位置 `ref` 为基础移动 `offset` 字节
`gcount()`	返回最后一次输入所读入的字节数

- **参照位置 (ref)**：
  - `ios::beg`：以文件**开始**为起点 (默认值)。
  - `ios::cur`：以指针**当前**位置为起点。
  - `ios::end`：以文件**结尾**为起点。

4. 字符串流

- (注：此类库在 `strstream` 头文件中，现代C++中已被 `<sstream>` 中的 `istringstream`，`ostringstream` 替代，但此处按教材内容记录。)

建立输出字符串流对象 (ostrstream)

1. **构造函数原型**：`ostrstream::ostrstream(char *bu, int n, int mode=ios::out);`
2. `bu` 是指向字符数组首址的指针，`n` 是流缓冲区的长度。
3. 作用：通过字符串流把数据**写入**字符数组 `bu`。
4. 示例：`ostrstream strout( ch1, 20);`

建立输入字符串流对象 (istrstream)

1. **构造函数原型**:

- `istrstream::istrstream(char *bu, int n);`
- `istrstream::istrstream(char *bu);`

1. `bu` 是指向字符数组首址的指针。
2. 作用：从字符数组 `bu` 中**读取**数据。
3. 示例: `istrstream strin(ch2);`

建立输入输出字符串流对象 (strstream)

1. **构造函数原型**: `strstream::strstream(char *bu, int n, int mode);`
2. 示例: `strstream strio(ch3, sizeof(ch3), ios::in | ios::out);`

第8章 编程规范

1. 头文件

头文件保护

- 所有头文件都应该使用 `#define` 来防止头文件被多重包含。
- 命名格式: `<PROJECT>_<PATH>_<FILE>_H_` 。
- 示例：项目 `foo` 中的头文件 `foo/src/bar/baz.h` :

```
#**ifndef** FOO_BAR_BAZ_H_
#**define** FOO_BAR_BAZ_H_
...
#**endif** // FOO_BAR_BAZ_H_
```

内联函数

- 只有当函数只有10行甚至更少时才会将其定义为内联函数。
- 内联那些包含循环或 `switch` 语句的函数是得不偿失的，除非在大多数情况下，这些循环或 `switch` 语句从不执行。
- 递归函数不应该被声明为内联。

函数参数顺序

- 定义函数时，参数顺序为：输入参数在前，输出参数在后。
- 输入参数一般传值或常数引用 (const references)。
- 输出参数或输入/输出参数为非常数指针 (non-const pointers) 或引用。
- 示例：
  - `void add(const int &a, const int &b, int *c)` (输入: a, b; 输出: c)
  - `void add(const int &a, const int &b, int &c)` (输入: a, b; 输出: c)

头文件包含顺序

- 将包含次序标准化可增强可读性、避免隐藏依赖。
- 次序如下：C库、C++库、其他库的.h、项目内的.h。
- 示例：`dir/foo.cc` （用于测试 `dir2/foo2.h` ）的包含次序：
  1. `dir2/foo2.h` (优先位置)
  2. C系统文件
  3. C++系统文件
  4. 其他库头文件
  5. 本项目内头文件

2. 变量作用域

局部变量

- 将函数变量尽可能置于最小作用域内。
- 尽可能靠近首次使用位置。
- 在声明变量时将其初始化。
  - 坏：`int i; i = f();` (声明和初始化分离)
  - 好：`int j = g();` (声明时初始化)
- 如果变量是一个对象，应将其声明在循环外，以避免构造函数和析G构函数被多次调用。
  - 坏 (调用1000000次构造和析构)

```
for (int i=0; i<1000000; ++i)
{
    Foo f;
```

```
    f.DoSomething(i);
}
```

- 好 (只调用1次):

```
Foo f;
for (int i=0; i<1000000; ++i)
{
    f.DoSomething(i);
}
```

全局变量

- `class` 类型的全局变量是被禁止的。
- 内建类型的全局变量是允许的。
- 多线程代码中，非常数全局变量也是被禁止的。
- 永远不要使用函数返回值初始化全局变量。
- 对于全局的字符串常量, 使用C风格的字符串，而不要使用STL的字符串。
  - `const char kFrogSays[] = "ribbet";`

3. 编程特性

降低耦合

- 一个函数只做一件事。
- 不要过早在意细节优化。
- 先用最清楚的逻辑描述出程序的框架。
- 减少全局变量的数量。
- 试图面向对象。

前置自增和自减

- 不考虑返回值的话, 前置自增 (`++i`) 通常要比后置自增 (`i++`) 效率更高。
- 因为后置的自增自减需要对表达式的值 `i` 进行一次拷贝。
- 如果 `i` 是迭代器或其他非数值类型, 拷贝的代价是比较大的。
- 对迭代器和模板类型来说, 要使用前置自增(自减)。

预处理宏

- 宏意味着你和编译器看到的代码是不同的, 因此可能导致异常行为。
- 使用宏时要谨慎, 尽量以内联函数、枚举和常量代替之。
  - 宏: `#define MAX 1024`
  - 代替: `const int MAX=1024;`
  - 宏: `#define ADD(a,b) (a+b)`
  - 代替: `inline int ADD(int a, int b){return a+b;}`

尽量用 `const` 和 `inline` 而不用 `#define`

- 尽量用编译器而不用预处理。
- 宏函数可能导致副作用，例如 `max(++a, b)`， `++a` 可能会被执行多次。
- 使用 `inline` 模板函数更安全。

尽量用 `<iostream>` 而不用 `<stdio.h>`

- `scanf` 和 `printf` 不是类型安全的, 而且没有扩展性。
- 类型安全和扩展性是C++的基石。

尽量用 `new` 和 `delete` 而不用 `malloc` 和 `free`

- `new` 和 `delete` 会调用构造函数和析构函数。
- 析构函数里应对指针成员调用 `delete` 。

0 和 NULL

- 整数用 `0` 。
- 实数用 `0.0` 。
- 指针用 `NULLPTR` (C++11 `nullptr`)。
- 字符(串)用 `'\0'` 。

4. 命名约定

- 最重要的一致性规则是命名管理。
- 命名风格能直接确定命名实体是类型、变量、函数、常量等，无需查找实体声明。
- 一致性比个人喜好更重要。团队合作应使用统一的约定。

通用命名规则

- 函数命名、变量命名、文件命名应具有描述性, 不要过度缩写。
- 类型和变量应该是名词。
- 函数名可以用“命令性”动词 (如 `OpenFile()`)。
- 尽量不使用缩写。
  - 推荐: `int num_completed_connections`
  - 不推荐: `int n` , `int n_comp_conns`

文件名

- 文件名要全部小写, 可以包含下划线 (`_`) 或短线 ()。
  - 如: `my_useful_class.cc` , `my-useful-class.cc`
- 尽量让文件名更加明确 (如 `http_server_logs.h` 优于 `logs.h`)。
- 定义类时文件名一般成对出现, 如类 `FooBar` 对应 `foo_bar.h` 和 `foo_bar.cpp` 。

类型命名

- 类型命名每个单词以大写字母开头 (PascalCase), 不包含下划线。
- 适用于所有类型：类、结构体、类型定义(typedef)、枚举。
- 示例: `MyExcitingClass` , `MyExcitingEnum` 。

变量命名

- 变量名一律小写, 单词间以下划线相连 (snake_case)。
  - 示例: `int my_exciting_local_variable`
- 类的**成员变量**以下划线结尾。
  - 示例: `int my_exciting_member_variable_`
- **全局变量**可以以 `g_` 为前缀。
  - 示例: `int g_stu_num`
- **常量** (编译时常量) 以 `k` 开头，后接大写字母开头的单词。
  - 示例: `const int kDaysInAWeek = 7;`

5. 格式 (与注释)

- 目标是保证代码可读性。

- "Code Tells You How, Comments Tell You Why" (代码告诉你怎么做，注释告诉你为什么这么做)。
- 风格: 使用 `//` 或 `/**/`, 保持统一即可。
- **文件注释**: 在每一个文件开头加入版权公告, 然后是文件内容描述。
- **类注释**: 每个类的定义要附着描述类的功能和用法的注释。
- **函数注释**:
  - **声明处**: 注释描述函数功能。
  - **定义处**: 注释描述函数实现。
- **代码注释**: 对于实现代码中巧妙的、晦涩的、有趣的、重要的地方加以注释。